



**University of
South Australia**



*COMP 1048: UO Object
Oriented Programming*

**ADVANCED PROGRAMMING AND
TESTING**

Copyright

This material has been reproduced and communicated to you by or on behalf of the University of South Australia under Section 113P of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the act.

Details

- Weighting **20%** of your overall grade.
- This assignment is to be completed **individually**.
- Due Week 10
- An object-oriented solution is required using PyCharm and Python programming.
- Maintain git version control through GitHub Classroom.

Course Objectives to Achieve

- | | |
|------|--|
| CO1. | Convert a problem statement into an object-oriented solution. (GQ1, GQ3) |
| CO2. | Use inheritance and polymorphism to solve problems. (GQ1, GQ3) |
| CO3. | Debug and fix code defects. (GQ1, GQ3) |
| CO4. | Apply international coding style standards. (GQ1, GQ4, GQ6) |
| CO5. | Plan and conduct testing of software. (GQ1, GQ3) |
| CO6. | Analyse and explain the behaviour of object-oriented programs. (GQ1, GQ3, GQ6) |

Academic Integrity

See Course Outline section: **Academic Integrity**. Artificial Intelligence (AI) tools are **prohibited** within this course and any detection of unauthorised AI will be reported as Academic Integrity.

Important Note

This document proposes the requirements for all components of the assessment. This document may contain imperfectly or potentially incorrectly written sections. If you spot any errors, or are uncertain of anything, please post on the discussion forum.

We bought a zoo!

Background: It is not every day that Simone decides to quit their job and fulfill their dream of owning and running the local zoo. Unfortunately Simone did not do all of their due diligence and failed to realise that the zoo was still run completely on pen and paper with records scattered all over the facility. Since Simone cannot pull out of the purchase they have contracted ByteWise Consulting (you) to prepare a computer system for them to manage the zoo.

Because this is a brand new custom system Simone has requested:

Core Requirements

Your Zoo Management System must support the management of animals, enclosures, and staff, using object-oriented principles to structure your solution.

The system should allow for the creation and management of animals, each with attributes such as name, species, age, and dietary needs. Animals may belong to different categories—such as mammals, reptiles, or birds—which may share common behaviors while also exhibiting unique traits. All animals should be capable of performing basic actions like making sounds, eating, and sleeping.

Enclosures are used to house animals and must include properties such as size, environmental type (e.g., aquatic, savannah), and cleanliness level. Each enclosure is restricted to a single type of animal, meaning incompatible species should not be housed together (e.g., koalas should not be placed in a penguin enclosure). Enclosures should be able to report their current status and list the animals they contain. Your system should ensure that animals are only placed in enclosures that are appropriate for their species and environmental needs.

Staff members play a key role in zoo operations and should be modeled with roles such as zookeeper or veterinarian, each with specific responsibilities. Staff should be able to perform actions including feeding animals, cleaning enclosures, and conducting health checks. They should also be assigned to specific animals or enclosures based on their role and duties.

Other features:

Beyond the core functionality, your system should support broader zoo operations. This includes the ability to add and remove animals, enclosures, and staff; assign animals to enclosures; schedule daily routines such as feeding and cleaning; and generate reports, such as lists of animals by species or the status of enclosures.

The system must also include functionality for documenting and managing animal health. This involves recording health issues such as injuries, illnesses, or behavioral concerns, along with relevant details including a description of the issue, the date it was reported, its severity level, and any treatment plans or notes. The system should be able to generate health reports for individual animals or across the zoo, and health status should influence zoo operations—for example, animals under treatment should not be moved or displayed.

Main.py:

Your Main.py file should act as a demonstration script. It does not need to be interactive, but it should clearly show how your system works by creating objects, invoking methods, and printing results. Think of it as a walkthrough of your zoo in action.

NOTE: There is not a single objectively correct solution and you will be assessed on how you have thought through your approach.

Tasks

Below is a list of the tasks you need to complete:

1. Design and Implement an Object-Oriented solution to the objective described above.
 1. Ensure that you are following the style taught throughout the course (camelCase, snake_case)
 2. Each class should be a separate file.
 3. Consider the visibility of your attributes and how the classes interact with each other.
 4. It is expected that you complete data validation on any supplied data to ensure it is valid.
2. Include unit tests with your code to show the correctness of your code.

Submission

Submit your design as a video and python code clearly identifying:

- Python Code:
 - Solution to the objective described above.
 - You will be expected to zip your repository and submit it.
- Video overview:
 - An 8–15 minute video demonstrating your solution and discussing the following questions:
 - How did you decide on the class hierarchy for animals, enclosures, and staff?
 - What trade-offs did you consider when choosing?
 - How did you model health records in your system?
 - Why did you choose to integrate them in that way, and what alternatives did you consider?
 - Did you encounter any challenges in maintaining encapsulation or separation of concerns?
 - How did you resolve them?
 - Have you added any extra functionality beyond the requirements above?
 - What?
 - Why?
 - Where did you apply polymorphism in your design?
 - How did it help simplify or generalize behavior?
 - Did you use interfaces or abstract classes?
 - If so, what role did they play in your design?
 - What was the most challenging part of this assignment for you?
 - How did you overcome it?
 - What did you learn about object-oriented programming through this project that you didn't fully understand before?

Marking Scheme

The marking allocation is split across the coded solution and the video reflection. The code is worth 50% and the reflection is worth 50% for this assignment.

The rubric for the **code**:

Criteria	Excellent (HD)	Distinction (D)	Credit (C)	Satisfactory (P)	F1	Fail (F2)
Class Design & Structure 10%	Well-structured class hierarchy with clear use of inheritance, encapsulation, and polymorphism. Design is modular and extensible.	Good use of OOP principles; class structure is logical, modular and extensible.	Good use of OOP principles; class structure is logical and mostly modular.	Reasonable class design; some use of OOP principles, though inconsistently applied.	Basic class structure; limited or superficial use of OOP principles.	Class design is missing, incoherent, or violates core OOP principles.
Functionality & Requirements 10%	All core features implemented correctly, including animal management, enclosures, staff, and health records. Functionality beyond the above requirements has been considered and implemented.	All core features implemented correctly, including animal management, enclosures, staff, and health records.	Minimum set of features have been implemented correctly and some extra features have been included.	Minimum set of features present and implemented	Minimal implementation of core features; significant gaps or errors.	Core functionality is largely missing or non-functional.
Code Quality & Style 10%	Code is clean, well-organized, and consistently follows style conventions. Includes meaningful comments and documentation.	Code is mostly clean and readable; some inconsistencies in style or documentation.		Code is readable but may be cluttered or inconsistently styled; limited documentation.	Code is difficult to read or poorly organized; lacks documentation.	Code is unreadable, disorganized, or lacks meaningful structure and documentation.

Use of Testing 10%	Comprehensive test coverage using appropriate testing tools or frameworks. Tests cover edge cases and validate key behaviors.	Good test coverage; tests validate most core functionality.	Basic tests included; coverage is limited or superficial.	Little to no testing; tests are missing or ineffective.	No testing present or testing code is non-functional.
Error Handling & Robustness 10%	System handles invalid input and edge cases gracefully; includes meaningful error messages and safeguards.	Some error handling present; system is mostly robust.	Basic error handling; system may crash or behave unpredictably in some cases.	Minimal or no error handling; system is fragile or unreliable.	No error handling; system fails under common or expected conditions.

The rubric for the **reflection**:

Criteria	Excellent (HD)	Good (D)	Credit (C)	Satisfactory (P)	Needs Improvement (F)
Design Rationale 10%	Clearly explains class hierarchy and design choices with strong justification and consideration of alternatives.	Explains design choices with some justification; alternatives briefly mentioned.	Provides a reasonable explanation of design choices; some awareness of alternatives.	Basic explanation of design choices; limited reflection on alternatives.	Minimal or unclear explanation of design decisions.
Health System Integration 10%	Thoughtful explanation of how health records were modeled and integrated; considers implications and alternatives.	Adequate explanation of health system design; some consideration of implications.	Describes health system integration with some clarity; limited reflection on impact.	Basic description of health system; limited reflection on design impact.	Health system poorly explained or missing.
OOP Principles 10%	Demonstrates deep understanding of OOP concepts (e.g., inheritance, polymorphism); clearly connects them to implementation.	Shows good understanding of OOP principles; connects them to code with minor gaps.	Demonstrates general understanding; some connection to implementation.	Demonstrates basic understanding; connections to code are superficial.	Limited or incorrect understanding of OOP principles.
Problem-Solving & Challenges 10%	Insightful reflection on challenges faced and strategies used to overcome them.	Describes challenges and solutions with some insight.	Mentions challenges and responses with basic reflection.	Mentions challenges but lacks depth in reflection.	Challenges not discussed or poorly articulated.
Personal Learning 10%	Clearly articulates personal growth and new understanding gained from the assignment.	Describes learning outcomes with some reflection.	Basic mention of learning; some evidence of personal insight.	Basic mention of learning; lacks depth.	Little to no reflection on learning.

Reviews

All submissions may be subject to a review to ensure academic integrity standards are upheld.

Extensions

All extension requests require a valid medical certificate or supporting documentation, or they will be declined without consideration. These must be received in advance of the due date. Extensions requested in the last days may be declined.